

RFP_LLM

The project aims to create a query-answering system for Requests for Proposals (RFPs). The system is designed to provide high-quality answers by leveraging the context of the RFP and the company profile. Additionally, the system can extract questions ending with a question mark and explore interesting topics from the RFP. The system also includes a command-line chatbot for interacting with the query-answering system. In the future, the system may include features for generating complete proposals in response to RFPs.

Table of Contents

- [Background](#)
- [Functionalities](#)
- [Accepted File Formats](#)
- [System Features](#)
- [Why not traditional method of LLM's query answering?](#)
- [System Structure](#)
- [Artifact Structure](#)
- [Usage](#)
- [Examples](#)
- [Future Directions](#)

Background

Requests for Proposals (RFPs), similar to Requests for Quotations (RFQs), are documents issued by organizations when seeking services or products from potential vendors. These documents outline the requirements, expectations, and evaluation criteria that the organization uses to assess proposals from vendors. RFPs are commonly used in industries such as government, technology, and construction to solicit detailed bids and solutions for complex projects. The RFP process typically involves multiple stakeholders and requires a thorough review of submitted proposals to ensure they meet the organization's objectives.

Given the complexity of RFPs, it is essential for companies to respond with precise and well-informed answers that align with the context of the RFP as well as the company's capabilities. Automating this process with a query-answering system can help organizations streamline the response process, reduce manual effort, and ensure higher accuracy and relevance in their proposals.

Functionalities

- **Extract Questions and Topics:** Extracts relevant questions and topics for writing proposals.
- **Query Answering:** Answer query (or question) based on RFP and company profile.
- **Chatbot Mode:** A chatbot designed to assist with RFP-related queries.

Accepted File Formats

- The system supports the following file formats for both **RFPs** and **company profiles**: **.doc**, **.docx**, **.pdf**, **.txt**, **.xlsx**, **.xls**, **.csv**

Note: Additional formats may be supported in the future as required.

System Features

- **Minimal human involvement:** The system is capable of transforming unstructured data into structured data with minimal human involvement. The only required human action is to provide the file paths for the RFPs and company profiles.
- **Manage files in any length:** The system is designed to handle files of any length and extract information from them, regardless of their size. This is made possible by the system's ability to summarize information from individual chunks and then create a final summary by consolidating all the chunk summaries.
- **Supports multiple industries:** The system is designed to support a variety of industries. The current framework is specifically built with flexibility in mind to accommodate this feature.
- **Supports multiple LLMs:** The system can support various LLMs, including ChatGPT, LLaMA, Gemini, and many others. This flexibility is made possible through the LangChain framework and the system's modular design.
- **Context-Aware Answering:** The system is able to deliver high-quality answers by leveraging the context of both the RFP and the company profile. This is achieved through the system's architecture and effective prompt engineering.

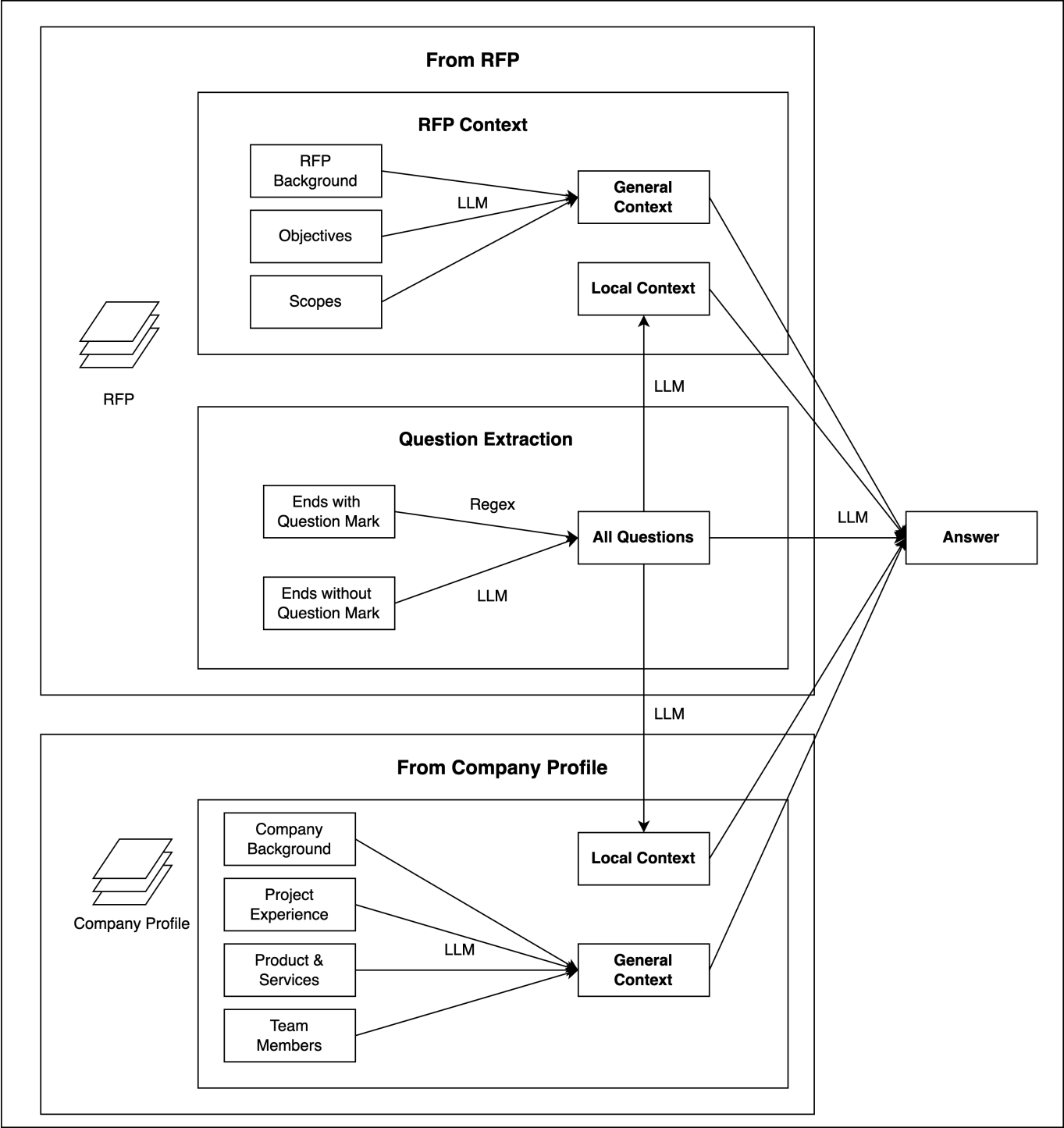
Why not traditional method of LLM's query answering?

The traditional method of query answering for RAG-LLM relies on chunking, vector stores, and retrievers. When a query is received, the system measures the semantic distance between the query and the available chunks. Then, the system retrieves multiple relevant chunks as context and inputs them into the LLM along with the query to generate an answer. The design of RAG effectively manages the token limits of LLMs, enabling the system to maximize the amount of context it can process while staying within the token limits.

However, the traditional RAG method has its limitations. First, when dealing with very large documents, RAG may only provide an answer using specific chunks, making it incapable of handling documents of any length effectively. Second, retrieving only particular chunks provides a limited context, failing to capture a more general context that could improve the quality and completeness of the answer. Third, the quality of the answer heavily depends on how well the retrieved chunks match the query. If the most relevant context is not among the retrieved chunks, the generated answer may be incomplete or inaccurate.

Our system aims to overcome the limitations of RAG. For chunk retrieval, we replace the traditional retriever with an LLM, using it to extract and compress the chunks iteratively. Our system leverages the LLM's capabilities in information retrieval, extraction, and compression, enabling it to provide context-aware answers while handling documents of any length with minimal human involvement. This forms the key advantage of our system.

System Structure



• **Question Extraction**

The system extracts questions ending with a question mark using Regular Expressions (Regex) and identifies interesting questions (topics) with the help of an LLM.

• **Query Answering**

The system leverages both the local context and general context from the RFP document and the company profile when generating an answer. **Local context** refers to query-specific information, while **general context** refers to a broad overview or description of the entire document.

• **Why general context?**

When designing the structure of the system, we need to understand that "context is an abstract concept." It is difficult to expect a machine (LLM) to provide an exact output that aligns with what we have in mind. For example, when extracting information for a query like "How to perform cybersecurity?", we know that software is an important context to consider. However, an LLM typically extracts information that explicitly or semi-explicitly mentions cybersecurity, and therefore may fail to extract related information, such as software.

It's important to recognize that as humans, we can identify the gap between the LLM's output and what we have in mind. To address this, we introduced the concept of "general context," which extracts broader information and provides a more comprehensive context for query-answering.

- **What should be included in the general context?**

- **It includes general and important information.**

The general context should provide a broad overview of the RFP or company profile, serving as an essential supplement to the local context. It covers features common to all RFP documents rather than specific sections.

- **It may not be extracted from local context.**

It is difficult to determine which information the LLM will extract and which it will not. Therefore, when selecting certain metrics, we should assess the likelihood of these metrics being extracted from the local context.

- **It is not industry-specific.**

Industry-specific information should be included in the general context. For example, software and cloud platforms are important contexts in the AI or software industries, while the location of a mine is important in the mining industry. We are considering creating a section dedicated to industry-specific context. However, at present, industry-specific context is embedded within the general context.

- **Why Background, Objectives, Scopes? (For RFP general context)**

Normally, the structure of an RFP consists of the following sections: 1) Project Overview 2) Scopes 3) Requirement 4) Specification 5) Budget, Pricing, Timeline 6) Evaluation Criteria 7) Terms and Conditions etc. We also noted that the sections may be varied for different RFP documents.

We ultimately chose background, objectives, and scopes. (Budget, pricing, and timeline are used as examples or instances in the prompts for objectives.) When considering other sections like requirements, specifications, and evaluation criteria, we found them to be too specific and better suited for extraction by the local context. They do not meet the criteria mentioned above and are therefore not included in the general context.

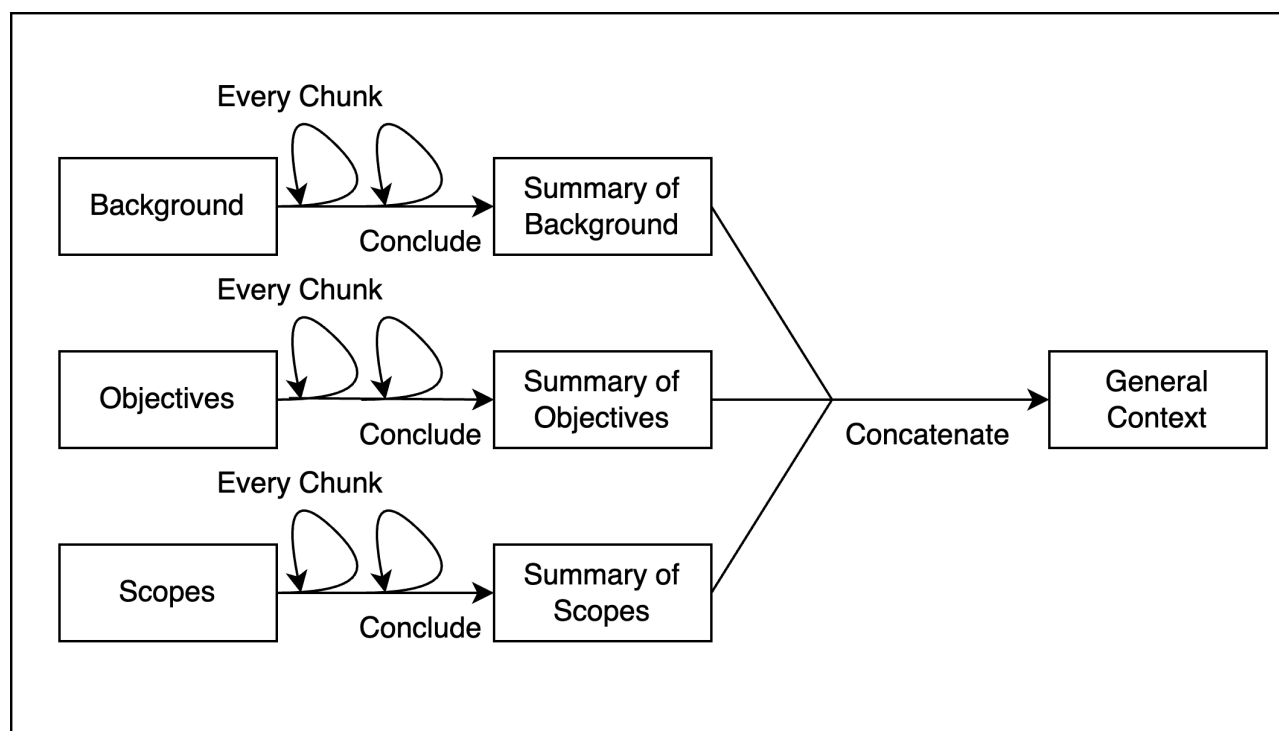
- **For company profile (general context)**

Likewise as above.

- **Note:** Please note that this is only a prototype system. Metrics may change as gain a better understanding of both the documents in the process of

developing the system.

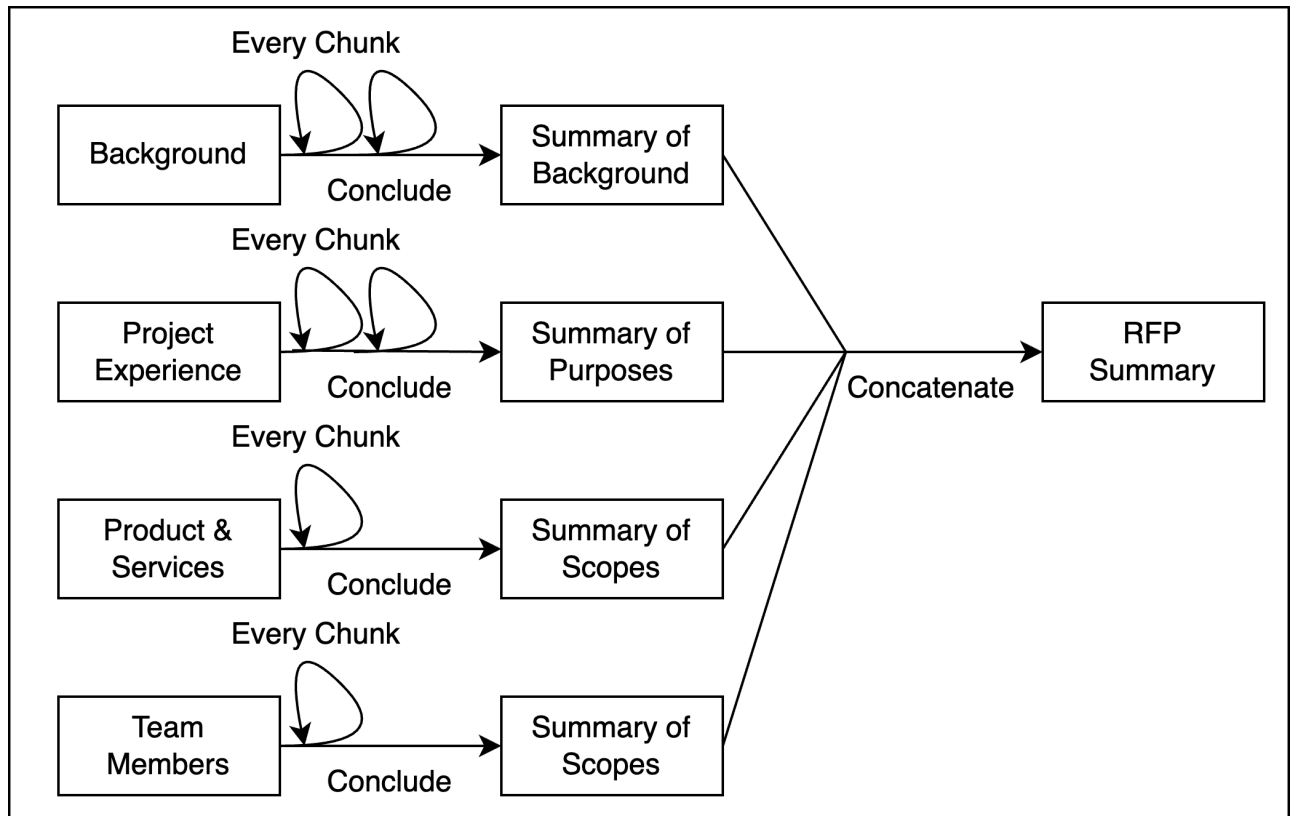
• RFP General Context



The RFP General Context is divided into 3 metrics: Background, Objectives, and Scopes.

- Each metric is summarized from individual chunks, and a final summary is created by consolidating all the chunk summaries.
- Each chunk is summarized **twice**. The first summary is used to locate the exact metric in the RFP by identifying the relevant heading in the document. The second is a general summarization of the chunk content.
- **Why Twice** for each chunk? We found that if we rely only on the first method (locating the heading), sometimes the exact heading doesn't exist in the RFP. For example, the background information might be found in the introduction section, and there may not be a specific "Background" section. On the other hand, if we use only the second method (general summarization), the LLM tends to give a broad summary for each subtopic without referring to the relevant section in the RFP.

- **Company Profile General Context**



The Company Profile General Context is divided into 4 metrics: Background, Project Experience, Product & Services, Team Members.

- Each metric is summarized from individual chunks, and a final summary is created by consolidating all the chunk summaries. (The same as RFP General Context)
- Each chunk in background and project experience is summarized twice`. The first summary is used to locate the exact metric in the RFP by identifying the relevant heading in the document. The second is a general summarization of the chunk content. (See **why Twice** above)
- The reason why Product & Service and Team Members only make a summary once for each chunk because they are not interweaving with other sections or concepts. For examples, objectives and scopes are interweaving concepts because they have similar definition.

Artifact Structure

- **data/**: Store RFPs and company profiles
- **meetings/**: Store previous meeting files
- **src/model_initializer.py**: To experiment with different versions of LLM models.
- **src/prompt_and_chain.py**: For prompts and chains
- **src/text_splitter.py**: To experiment with different versions of text splitters.
- **src/timer.py**: A timer for chatbot when executing query
- **src/utlis.py**: Contains utility functions, including file reading and regex matching.
- **src/vectorstore_retriever.py**: Implements different vector store and retriving methods for retrieval. (Deprecated)
- **Chatbot.py**: The interface of the chatbot
- **config.py**: Configuration of ROP paths and LLM keys
- **key.py**: LLM keys (.gitignore)

- [RAG_chain.py](#): Data preparation and query-answering chain (The Class that links everything together.)
- [main.py](#): Main function to execute functionalities

Usage

Functions are all well encapsulated.

- **LLM keys**: Put your LLM key(s) into [key.py](#).
- **document path**: Put your RPF document(s) and company profile document(s) path into [config.py](#).
- **main.py**: Create an instance of RAG_Chain.
- **Chatbot.py (optional)**: Launch Chatbot mode, put your instance of RAG_Chain into your instance of Chatbot.

Examples

- Reference: [.examples/example.ipynb](#), [.examples/test.ipynb](#)

Future Directions

- **Provides more details in the answer**

The system is now capable of giving a general answer that highlights key points. It is also possible to explore these key points in greater detail.

- **Manage files in any length**

We're almost there, but there are still a few more steps to go before we can fully manage files of any length. Specifically, improvements are needed in the chunking summary process

- **Industry-related Context**

Storing the industry-related context that demands to be extracted in the [./data/industry_context](#). The system can extract industry-related context from RFPs or company profile with minimal human intervention.